

TODO V1 — Répartition du travail (binôme:ETU004311; ETU003968)

Principe d'organisation

- **Branche:dev_misaina_login_auto — Côté Opérateur** (admin/back-office)
- **Brache:dev_toavina_client — Côté Client** (login + opérations)

Chacun lit dans les memes tables (`clients`, `prefixes`, `types_operation`, `frais`, `transactions`), deja creees par `base.sql` — aucun besoin de se coordonner sur le schema, il est fige.

Workflow git suggere :

- Une branche par partie (ex: `feature/operateur`, `feature/client`), partant de `main` a jour
- Chacun push sa branche + PR vers `dev` quand sa partie est testee
- Seuls `app/Config/Routes.php` et `app/Views/layouts/main.php` sont partagés (voir section Coordination) : petits fichiers, conflits faciles a resoudre a la main

Lot 1 — Côté Opérateur (ETU003968)Termine

Fichiers créés (propres à ce lot, aucun croisement avec le Lot 2) :

```
app/Models/PrefixModel.php          (nom reel : PrefixModel, sans "e" - deja cree)
app/Models/TypeOperationModel.php
app/Models/FraisModel.php
app/Controllers/OperateurController.php
app/Views/operateur/prefixes.php
app/Views/operateur/types_operation.php
app/Views/operateur/frais.php        (partial inclus par types_operation.php)
app/Views/operateur/comptes.php
app/Views/operateur/gains.php
app/Views/operateur/_flash.php       (partial messages succes/erreur, reutilise partout)
```

Checklist et code :

- ☒ `PrefixModel` — CRUD simple sur `prefixes` (table: `id`, `prefixe`)

`app/Models/PrefixModel.php`

```
<?php

namespace App\Models;

use CodeIgniter\Model;
```

```
class PrefixModel extends Model
{
    protected $table = 'prefixes';
    protected $primaryKey = 'id';

    protected $allowedFields = ['prefixe'];
    protected $useTimestamps = false;
}
```

- ☒ **TypeOperationModel** — CRUD sur `types_operation` (table: `id`, `code`, `libelle`)

app/Models/TypeOperationModel.php

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class TypeOperationModel extends Model
{
    protected $table = 'types_operation';
    protected $primaryKey = 'id';

    protected $allowedFields = ['code', 'libelle'];
    protected $useTimestamps = false;
}
```

- ☒ **FraisModel** — CRUD + `pourType($typeOperationId)` sur `frais` (table: `id`, `type_operation_id`, `min`, `max`, `valeur`)

app/Models/FraisModel.php

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class FraisModel extends Model
{
    protected $table = 'frais';
    protected $primaryKey = 'id';

    protected $allowedFields = ['type_operation_id', 'min', 'max', 'valeur'];
    protected $useTimestamps = false;

    /** tri croissant */
}
```

```

    public function pourType(int $typeOperationId): array
    {
        return $this->where('type_operation_id', $typeOperationId)
            ->orderBy('min', 'ASC')
            ->findAll();
    }
}

```

- ☒ **OpérateurController** — `prefixes()` / `storePrefixe()` / `deletePrefixe()` (GET liste + formulaire, POST ajout avec validation format+unicite, POST suppression), `typesOperation()` / `updateFrais()` (GET barème par type, POST modification d'une tranche), `comptes()` / `gains()` (lecture directe des tables `clients` et `transactions` via `Database::connect()`, sans créer `ClientModel/TransactionModel` pour ne pas empiéter sur le Lot 2)

`app/Controllers/OpérateurController.php`

```

<?php

namespace App\Controllers;

use App\Models\FraisModel;
use App\Models\PrefixModel;
use App\Models\TypeOperationModel;
use Config\Database;

class OpérateurController extends BaseController
{
    protected PrefixModel $prefixeModel;
    protected TypeOperationModel $typeOperationModel;
    protected FraisModel $fraisModel;

    public function __construct()
    {
        $this->prefixeModel = new PrefixModel();
        $this->typeOperationModel = new TypeOperationModel();
        $this->fraisModel = new FraisModel();
    }

    public function prefixes()
    {
        return view('opérateur/prefixes', [
            'prefixes' => $this->prefixeModel->orderBy('prefixe', 'ASC')->findAll(),
        ]);
    }

    public function storePrefixe()
    {
        $rules = [

```

```
        'prefixe' => 'required|regex_match[/^[0-9]{2,5}$/]|is_unique[prefixes.prefixe]',
    ];

    if (! $this->validate($rules)) {
        return redirect()->to('opérateur/prefixes')->withInput()->with('errors', $this->validator->getErrors());
    }

    $this->prefixeModel->insert(['prefixe' => $this->request->getPost('prefixe')]);

    return redirect()->to('opérateur/prefixes')->with('success', 'Prefixe ajouté.');
```

```
    }

    public function deletePrefixe(int $id)
    {
        $this->prefixeModel->delete($id);

        return redirect()->to('opérateur/prefixes')->with('success', 'Prefixe supprimé.');
```

```
    }

    public function typesOperation()
    {
        $types = $this->typeOperationModel->findAll();

        foreach ($types as &$type) {
            $type['frais'] = $this->fraisModel->pourType((int) $type['id']);
        }
        unset($type);

        return view('opérateur/types_operation', ['types' => $types]);
    }

    public function updateFrais(int $id)
    {
        $rules = [
            'min'    => 'required|numeric',
            'max'    => 'required|numeric|greater_than[{min}]',
            'valeur' => 'required|numeric',
        ];

        if (! $this->validate($rules)) {
            return redirect()->to('opérateur/types-operation')->with('errors', $this->validator->getErrors());
        }

        $this->fraisModel->update($id, [
            'min'    => $this->request->getPost('min'),
            'max'    => $this->request->getPost('max'),
```

```

        'valeur' => $this->request->getPost('valeur'),
    ]);

    return redirect()->to('opérateur/types-operation')->with('success',
'Bareme mis a jour.');
```

```

    public function comptes()
    {
        $clients = Database::connect()->table('clients')
            ->orderBy('solde', 'DESC')
            ->get()
            ->getResultArray();

        return view('opérateur/comptes', ['clients' => $clients]);
    }

    public function gains()
    {
        $db = Database::connect();

        $gains = $db->table('transactions')
            ->select('types_operation.libelle AS libelle,
SUM(transactions.frais) AS total_frais, COUNT(transactions.id) AS
nb_operations')
            ->join('types_operation', 'types_operation.id =
transactions.type_operation_id')
            ->groupBy('types_operation.id')
            ->get()
            ->getResultArray();

        $totalGeneral = array_sum(array_column($gains, 'total_frais'));

        return view('opérateur/gains', [
            'gains' => $gains,
            'totalGeneral' => $totalGeneral,
        ]);
    }
}

```

- ☒ Vues Bootstrap (tableaux + formulaires) qui étendent `layouts/main`, testées en HTTP (200 sur les 4 pages, ajout/suppression préfixe + modification frais vérifiés)

`app/Views/opérateur/_flash.php` (partial messages succes/erreur, reutilise partout)

```

<?php if (session()->getFlashdata('success')) : ?>
    <div class="alert alert-success"><?= esc(session()-
>getFlashdata('success')) ?></div>
<?php endif; ?>

```

```

<?php if (session()->getFlashdata('errors')) : ?>
    <div class="alert alert-danger">
        <ul class="mb-0">
            <?php foreach (session()->getFlashdata('errors') as $error) : ?>
                <li><?= esc($error) ?></li>
            <?php endforeach; ?>
        </ul>
    </div>
<?php endif; ?>

```

app/Views/operateur/prefixes.php

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<h1 class="h4 mb-4">Prefixes valables</h1>

<?= $this->include('operateur/_flash') ?>

<div class="row">
    <div class="col-md-6">
        <table class="table table-bordered bg-white">
            <thead>
                <tr>
                    <th>Prefixe</th>
                    <th class="text-end">Action</th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($prefixes as $prefixe) : ?>
                    <tr>
                        <td><?= esc($prefixe['prefixe']) ?></td>
                        <td class="text-end">
                            <form method="post" action="<?=
site_url('operateur/prefixes/' . $prefixe['id'] . '/delete') ?>"
                                onsubmit="return confirm('Supprimer ce
prefixe ?');">
                                <?= csrf_field() ?>
                                <button type="submit" class="btn btn-sm btn-
outline-danger">Supprimer</button>
                            </form>
                        </td>
                    </tr>
                <?php endforeach; ?>
                <?php if (empty($prefixes)) : ?>
                    <tr>
                        <td colspan="2" class="text-muted">Aucun prefixe
configure.</td>
                    </tr>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

```

```

        </table>
    </div>

    <div class="col-md-6">
        <div class="card">
            <div class="card-body">
                <h2 class="h6">Ajouter un prefixe</h2>
                <form method="post" action="{?=
site_url('opérateur/prefixes') ?}>">
                    <?= csrf_field() ?>
                    <div class="mb-3">
                        <label class="form-label" for="prefixe">Prefixe (ex:
033)</label>
                        <input type="text" class="form-control" id="prefixe"
name="prefixe"
                                value="{?= esc(old('prefixe')) ?}"
maxlength="5" required>
                    </div>
                    <button type="submit" class="btn btn-
primary">Ajouter</button>
                </form>
            </div>
        </div>
    </div>
</div>
<?= $this->endSection() ?>

```

app/Views/opérateur/types_operation.php

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<h1 class="h4 mb-4">Types d'opération & bareme de frais</h1>

<?= $this->include('opérateur/_flash') ?>

<?php foreach ($types as $type) : ?>
    <?= $this->setVar('type', $type)->include('opérateur/frais') ?>
<?php endforeach; ?>
<?= $this->endSection() ?>

```

app/Views/opérateur/frais.php (partial inclus par types_operation.php, un formulaire Bootstrap par ligne du barème — <form> en grille plutôt qu'imbriqué dans un <table> pour rester en HTML valide)

```

<div class="card mb-4">
    <div class="card-header">
        <strong><?= esc($type['libelle']) ?></strong>
        <span class="text-muted">(<?= esc($type['code']) ?>)</span>
    </div>

```

```

</div>
<div class="card-body">
    <?php if (empty($type['frais'])) : ?>
        <p class="text-muted mb-0">Aucun frais applicable (operation
gratuite).</p>
    <?php else : ?>
        <div class="row fw-bold small text-muted mb-2">
            <div class="col-3">Montant min</div>
            <div class="col-3">Montant max</div>
            <div class="col-3">Frais</div>
            <div class="col-3"></div>
        </div>
        <?php foreach ($type['frais'] as $ligne) : ?>
            <form method="post" action="<?= site_url('opérateur/frais/' .
$ligne['id']) ?>" class="row g-2 align-items-center mb-2">
                <?= csrf_field() ?>
                <div class="col-3">
                    <input type="number" step="0.01" class="form-control
form-control-sm" name="min" value="<?= esc($ligne['min']) ?>">
                </div>
                <div class="col-3">
                    <input type="number" step="0.01" class="form-control
form-control-sm" name="max" value="<?= esc($ligne['max']) ?>">
                </div>
                <div class="col-3">
                    <input type="number" step="0.01" class="form-control
form-control-sm" name="valeur" value="<?= esc($ligne['valeur']) ?>">
                </div>
                <div class="col-3 text-end">
                    <button type="submit" class="btn btn-sm btn-outline-
primary">Enregistrer</button>
                </div>
            </form>
        <?php endforeach; ?>
    <?php endif; ?>
</div>
</div>

```

app/Views/opérateur/comptes.php

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<h1 class="h4 mb-4">Situation des comptes clients</h1>

<table class="table table-bordered bg-white">
    <thead>
        <tr>
            <th>Numero</th>
            <th class="text-end">Solde</th>
        </tr>
    </thead>

```



```

<tbody>
  <?php foreach ($clients as $client) : ?>
    <tr>
      <td><?= esc($client['numero']) ?></td>
      <td class="text-end"><?= number_format((float)
$client['solde'], 2, ',', ' ') ?> Ar</td>
    </tr>
  <?php endforeach; ?>
  <?php if (empty($clients)) : ?>
    <tr>
      <td colspan="2" class="text-muted">Aucun client pour le
moment.</td>
    </tr>
  <?php endif; ?>
</tbody>
</table>
<?= $this->endSection() ?>

```

app/Views/operateur/gains.php

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<h1 class="h4 mb-4">Situation des gains (frais percus)</h1>

<table class="table table-bordered bg-white">
  <thead>
    <tr>
      <th>Type d'operation</th>
      <th class="text-end">Nb operations</th>
      <th class="text-end">Total frais percus</th>
    </tr>
  </thead>
  <tbody>
    <?php foreach ($gains as $ligne) : ?>
      <tr>
        <td><?= esc($ligne['libelle']) ?></td>
        <td class="text-end"><?= (int) $ligne['nb_operations'] ?>
</td>
        <td class="text-end"><?= number_format((float)
$ligne['total_frais'], 2, ',', ' ') ?> Ar</td>
      </tr>
    <?php endforeach; ?>
    <?php if (empty($gains)) : ?>
      <tr>
        <td colspan="3" class="text-muted">Aucune operation payante
enregistree.</td>
      </tr>
    <?php endif; ?>
  </tbody>
<tfoot>
  <tr class="fw-bold">

```

```

        <td colspan="2">Total general</td>
        <td class="text-end"><?= number_format((float) $totalGeneral, 2,
        ', ', ' ') ?> Ar</td>
    </tr>
</tfoot>
</table>
<?= $this->endSection() ?>

```

Lot 2 — Côté Client (ETU004311) Termine

Fichiers créés (propres à ce lot, aucun croisement avec le Lot 1) :

```

app/Models/ClientModel.php
app/Models/TransactionModel.php
app/Controllers/AuthController.php
app/Controllers/ClientController.php
app/Filters/ClientAuthFilter.php
app/Views/auth/login.php
app/Views/client/dashboard.php
app/Views/client/depot.php
app/Views/client/retrait.php
app/Views/client/transfert.php
app/Views/client/historique.php

```

Checklist et code :

- ☒ **ClientModel** — CRUD sur **clients** (**id**, **numero**, **solde**), validation integree (numero unique, solde decimal) + **findByNumero()**

app/Models/ClientModel.php

```

<?php

namespace App\Models;

use CodeIgniter\Model;

class ClientModel extends Model
{
    protected $table          = 'clients';
    protected $primaryKey     = 'id';
    protected $returnType     = 'object';
    protected $useTimestamps = true;
    protected $createdField   = 'created_at';
    protected $updatedField   = 'updated_at';

    protected $allowedFields = [
        'numero',

```

```

        'solde',
    ];

    protected $validationRules = [
        'numero' => 'required|is_unique[clients.numero,id]',
        'solde' => 'required|decimal',
    ];

    protected $validationMessages = [
        'numero' => [
            'required' => 'Le numero de telephone est obligatoire.',
            'is_unique' => 'Ce numero de telephone est deja utilise.',
        ],
        'solde' => [
            'required' => 'Le solde est obligatoire.',
            'decimal' => 'Le solde doit etre un nombre decimal.',
        ],
    ];

    public function findByNumero(string $numero): ?object
    {
        return $this->where('numero', $numero)->first();
    }

    /**
     * Tous les clients sauf celui donne (suggestions de destinataire pour un
     * transfert).
     */
    public function findAllExcept(int $excludeId): array
    {
        return $this->where('id !=', $excludeId)->findAll();
    }
}

```

- ☒ `AuthController::login()` — GET, formulaire de saisie du numéro
- ☒ `AuthController::attempt()` — POST `login`, vérifie le préfixe (table `prefixes`, en lecture seule via `db_connect()`), crée le client s'il n'existe pas encore (login auto, aucune inscription), stocke `client_id` en session, redirige vers `client`
- ☒ `AuthController::logout()` — GET, détruit la session, redirige vers /

`app/Controllers/AuthController.php`

```

<?php

namespace App\Controllers;

use App\Models\ClientModel;

class AuthController extends BaseController

```

```
{

    public function login(): string
    {
        return view('auth/login');
    }

    public function attempt()
    {
        $numero = trim($this->request->getPost('numero') ?? '');

        if ($numero === '') {
            return redirect()->back()->withInput()->with('error', 'Veuillez
saisir un numero de telephone.');
```

```
        }

        $prefixe = substr($numero, 0, 3);

        $prefixeValide = db_connect()->table('prefixes')
            ->where('prefixe', $prefixe)
            ->countAllResults() > 0;

        if (! $prefixeValide) {
            return redirect()->back()->withInput()->with('error', 'Le prefixe
'' . esc($prefixe) . '' n\'est pas valable.');
```

```
        }

        $clientModel = new ClientModel();
        $client = $clientModel->findByNumero($numero);

        if ($client === null) {
            $clientModel->insert([
                'numero' => $numero,
                'solde'  => 0,
            ]);
            $client = $clientModel->findByNumero($numero);
        }

        session()->set('client_id', $client->id);

        return redirect()->to(site_url('client'));
    }

    public function logout()
    {
        session()->destroy();

        return redirect()->to(site_url('/'));
    }
}
```

- ☒ **ClientAuthFilter** — protège les routes `/client/*` (redirige vers login si pas de session), activé dans `Routes.php` (`$routes->group('client', ['filter' => 'clientAuth'], ...)`) et enregistré dans `app/Config/Filters.php` (`'clientAuth' => \App\Filters\ClientAuthFilter::class`)

`app/Filters/ClientAuthFilter.php`

```
<?php

namespace App\Filters;

use CodeIgniter\Filters\FilterInterface;
use CodeIgniter\HTTP\RequestInterface;
use CodeIgniter\HTTP\ResponseInterface;

class ClientAuthFilter implements FilterInterface
{
    public function before(RequestInterface $request, $arguments = null)
    {
        if (! session()->get('client_id')) {
            return redirect()->to(site_url('login'));
        }
    }

    /**
     * Aucune action apres la requete.
     */
    public function after(RequestInterface $request, ResponseInterface $response, $arguments = null)
    {
    }
}
```

- ☒ **TransactionModel::calculerFrais()** — lit le barème dans `frais` (lecture seule), voir aussi l'exemple de code dans [GUIDE_TECHNIQUE.md §5](#). `depot()` / `retrait()` / `transfert()` encapsulent chacune la mise a jour du/des solde(s) + l'insertion dans `transactions` dans une transaction SQL (`transStart()/transComplete()`), avec verification du solde et `throw` en cas de solde insuffisant. `getHistorique()` liste les operations d'un client.

`app/Models/TransactionModel.php`

```
<?php

namespace App\Models;

use CodeIgniter\Model;

class TransactionModel extends Model
{
```

```
protected $table      = 'transactions';
protected $returnType = 'object';

protected $allowedFields = [
    'type_operation_id',
    'client_id',
    'client_destination_id',
    'montant',
    'frais',
    'solde_avant',
    'solde_apres',
];

public function calculerFrais(int $typeOperationId, float $montant):
float
{
    $bareme = $this->db->table('frais')
        ->where('type_operation_id', $typeOperationId)
        ->where('min <=', $montant)
        ->where('max >=', $montant)
        ->get()
        ->getRow();

    return $bareme->valeur ?? 0.0;
}

public function depot(int $clientId, float $montant): array
{
    $clientModel = model(ClientModel::class);

    $this->db->transStart();

    $client      = $clientModel->find($clientId);
    $soldeApres = $client->solde + $montant;

    $clientModel->update($clientId, ['solde' => $soldeApres]);

    $this->insert([
        'type_operation_id' => $this->getCodeId('depot'),
        'client_id'         => $clientId,
        'montant'           => $montant,
        'frais'             => 0.0,
        'solde_avant'       => $client->solde,
        'solde_apres'       => $soldeApres,
    ]);

    $this->db->transComplete();

    return ['frais' => 0.0, 'solde' => $soldeApres];
}
```

```
public function retrait(int $clientId, float $montant): array
{
    $clientModel = model(ClientModel::class);

    $this->db->transStart();

    $client = $clientModel->find($clientId);
    $typeId = $this->getCodeId('retrait');
    $frais = $this->calculerFrais($typeId, $montant);
    $total = $montant + $frais;

    if ($client->solde < $total) {
        $this->db->transRollback();
        throw new \RuntimeException('Solde insuffisant.');
```

```
    }

    $soldeApres = $client->solde - $total;

    $clientModel->update($clientId, ['solde' => $soldeApres]);

    $this->insert([
        'type_operation_id' => $typeId,
        'client_id'         => $clientId,
        'montant'           => $montant,
        'frais'             => $frais,
        'solde_avant'       => $client->solde,
        'solde_apres'       => $soldeApres,
    ]);

    $this->db->transComplete();

    return ['frais' => $frais, 'solde' => $soldeApres];
}

public function transfert(int $clientId, int $clientDestinationId, float
$montant): array
{
    $clientModel = model(ClientModel::class);

    $this->db->transStart();

    $emetteur = $clientModel->find($clientId);
    $destinataire = $clientModel->find($clientDestinationId);
    $typeId = $this->getCodeId('transfert');
    $frais = $this->calculerFrais($typeId, $montant);
    $total = $montant + $frais;

    if ($emetteur->solde < $total) {
        $this->db->transRollback();
        throw new \RuntimeException('Solde insuffisant pour le
transfert.');
```

```

        $soldeEmetteur      = $emetteur->solde - $total;
        $soldeDestinataire = $destinataire->solde + $montant;

        $clientModel->update($clientId, ['solde' => $soldeEmetteur]);
        $clientModel->update($clientDestinationId, ['solde' =>
$soldeDestinataire]);

        $this->insert([
            'type_operation_id'      => $typeId,
            'client_id'              => $clientId,
            'client_destination_id'  => $clientDestinationId,
            'montant'                => $montant,
            'frais'                  => $frais,
            'solde_avant'            => $emetteur->solde,
            'solde_apres'            => $soldeEmetteur,
        ]);

        $this->db->transComplete();

        return ['frais' => $frais, 'solde' => $soldeEmetteur];
    }

    public function getHistorique(int $clientId): array
    {
        return $this->select('transactions.*, types_operation.libelle AS
type_libelle')
            ->join('types_operation', 'types_operation.id =
transactions.type_operation_id')
            ->where('client_id', $clientId)
            ->orderBy('created_at', 'DESC')
            ->findAll();
    }

    /**
     * Numeros des destinataires deja utilises par ce client dans ses
     * transferts,
     * du plus recent au plus ancien (sans doublon).
     */
    public function getDestinatairesRecents(int $clientId): array
    {
        $rows = $this->select('clients.numero')
            ->join('clients', 'clients.id =
transactions.client_destination_id')
            ->where('transactions.client_id', $clientId)
            ->where('transactions.client_destination_id IS NOT NULL')
            ->orderBy('transactions.created_at', 'DESC')
            ->findAll();

        return array_values(array_unique(array_map(static fn ($row) => $row-
>numero, $rows)));
    }

    private function getCodeId(string $code): int

```



```

    {
        $row = $this->db->table('types_operation')
            ->where('code', $code)
            ->get()
            ->getRow();

        return (int) $row->id;
    }
}

```

- ☒ `ClientController::dashboard()` — GET `client`, affiche le solde du client connecté
- ☒ `ClientController::depot()` / `storeDepot()` — GET formulaire / POST `client/depot`, crédite le solde, frais = 0, insère dans `transactions`
- ☒ `ClientController::retrait()` / `storeRetrait()` — GET formulaire / POST `client/retrait`, vérifie solde suffisant (montant + frais), débite, insère dans `transactions`
- ☒ `ClientController::transfert()` / `storeTransfert()` — GET formulaire / POST `client/transfert`, vérifie solde suffisant, débite l'émetteur, crédite le destinataire (`client_destination_id`), insère dans `transactions`
- ☒ `ClientController::historique()` — GET `client/historique`, liste des `transactions` du client connecté

`app/Controllers/ClientController.php`

```

<?php

namespace App\Controllers;

use App\Models\ClientModel;
use App\Models\TransactionModel;

class ClientController extends BaseController
{
    private function clientId(): int
    {
        return (int) session()->get('client_id');
    }

    public function dashboard()
    {
        $clientModel = new ClientModel();
        $client = $clientModel->find($this->clientId());

        return view('client/dashboard', ['client' => $client]);
    }
}

```

```
public function depot()
{
    return view('client/depot');
}

public function storeDepot()
{
    $montant = (float) $this->request->getPost('montant');

    if ($montant <= 0) {
        return redirect()->back()->with('error', 'Le montant doit etre
superieur a 0.');
```

```
    }

    $transactionModel = new TransactionModel();
    $transactionModel->depot($this->clientId(), $montant);

    return redirect()->to(site_url('client'))->with('success', 'Depot de
' . number_format($montant, 0, ',', ' ') . ' effectue.');
```

```
    }

public function retrait()
{
    return view('client/retrait');
}

public function storeRetrait()
{
    $montant = (float) $this->request->getPost('montant');

    if ($montant <= 0) {
        return redirect()->back()->with('error', 'Le montant doit etre
superieur a 0.');
```

```
    }

    $transactionModel = new TransactionModel();

    try {
        $resultat = $transactionModel->retrait($this->clientId(),
$montant);
    } catch (\RuntimeException $e) {
        return redirect()->back()->with('error', $e->getMessage());
    }

    return redirect()->to(site_url('client'))->with(
        'success',
        'Retrait de ' . number_format($montant, 0, ',', ' ') . ' effectue
(frais : ' . number_format($resultat['frais'], 0, ',', ' ') . ').'
```

```
    );
}
```

```
/**
 * Suggestions de numeros : destinataires deja utilises par ce client en
 * priorite,
 * puis les autres clients de la base.
 */
public function transfert()
{
    $clientModel = new ClientModel();
    $transactionModel = new TransactionModel();

    $recents = $transactionModel->getDestinatairesRecents($this->clientId());
    $autres = array_map(
        static fn ($client) => $client->numero,
        $clientModel->findAllExcept($this->clientId())
    );

    $suggestions = array_values(array_unique(array_merge($recents, $autres)));

    return view('client/transfert', ['suggestions' => $suggestions]);
}

public function storeTransfert()
{
    $destinataire = trim($this->request->getPost('destinataire') ?? '');
    $montant = (float) $this->request->getPost('montant');

    if ($destinataire === '') {
        return redirect()->back()->with('error', 'Veuillez saisir le
numero du destinataire.');
```

```
    }

    if ($montant <= 0) {
        return redirect()->back()->with('error', 'Le montant doit etre
superieur a 0.');
```

```
    }

    $clientModel = new ClientModel();
    $dest = $clientModel->findByNumero($destinataire);

    if ($dest === null) {
        return redirect()->back()->with('error', 'Le numero "' .
esc($destinataire) . '" est introuvable.');
```

```
    }

    if ((int) $dest->id === $this->clientId()) {
        return redirect()->back()->with('error', 'Vous ne pouvez pas vous
transférer a vous-meme.');
```

```
    }

    $transactionModel = new TransactionModel();
```

```

        try {
            $resultat = $transactionModel->transfert($this->clientId(), (int)
$dest->id, $montant);
        } catch (\RuntimeException $e) {
            return redirect()->back()->with('error', $e->getMessage());
        }

        return redirect()->to(site_url('client'))->with(
            'success',
            'Transfert de ' . number_format($montant, 0, ',', ' ') . '
effectue (frais : ' . number_format($resultat['frais'], 0, ',', ' ') . ').'
        );
    }

    public function historique()
    {
        $transactionModel = new TransactionModel();
        $transactions = $transactionModel->getHistorique($this->clientId());

        return view('client/historique', ['transactions' => $transactions]);
    }
}

```

- ☒ Vues Bootstrap (formulaires + tableaux) qui étendent `layouts/main`, testées en HTTP (login → dépôt → retrait → transfert → historique vérifiés de bout en bout, calcul des frais correct)

app/Views/auth/login.php

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<div class="d-flex flex-column justify-content-center align-items-center"
style="min-height: 60vh;">
    <div class="card shadow-sm" style="max-width: 400px; width: 100%;">
        <div class="card-body p-4">
            <h1 class="h4 mb-3 text-center">Connexion</h1>

            <?php $error = session()->getFlashdata('error'); ?>
            <?php if ($error): ?>
                <div class="alert alert-danger"><?= esc($error) ?></div>
            <?php endif; ?>

            <form method="post" action="<?= site_url('login') ?>">
                <div class="mb-3">
                    <label for="numero" class="form-label">Numero de
telephone</label>

                    <input
                        type="text"
                        class="form-control"
                        id="numero"

```

```

        name="numero"
        placeholder="Ex : 0331234567"
        value="<?= esc(old('numero')) ?>"
        required
    >
</div>
<button type="submit" class="btn btn-primary w-100">Se
connector</button>
</form>
</div>
</div>
</div>
<?= $this->endSection() ?>

```

app/Views/client/dashboard.php

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<div class="row justify-content-center">
    <div class="col-md-6">

        <?php $success = session()->getFlashdata('success'); ?>
        <?php if ($success): ?>
            <div class="alert alert-success"><?= esc($success) ?></div>
        <?php endif; ?>

        <div class="card shadow-sm">
            <div class="card-body text-center p-4">
                <h1 class="h4 mb-3">Mon solde</h1>
                <p class="text-muted">Numero : <?= esc($client->numero) ?>
            </p>
                <p class="display-5 fw-bold text-success">
                    <?= number_format($client->solde, 2, ',', ' ') ?> Ar
                </p>
            </div>
        </div>
    </div>
</div>
<?= $this->endSection() ?>

```

app/Views/client/depot.php

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<div class="row justify-content-center">
    <div class="col-md-6">

```

```

<?php $error = session()->getFlashdata('error'); ?>
<?php if ($error): ?>
    <div class="alert alert-danger"><?= esc($error) ?></div>
<?php endif; ?>

<div class="card shadow-sm">
    <div class="card-body p-4">
        <h1 class="h4 mb-3">Depot</h1>
        <form method="post" action="<?= site_url('client/depot') ?>">
            <?= csrf_field() ?>
            <div class="mb-3">
                <label for="montant" class="form-label">Montant (Ar)
</label>

                <input
                    type="number"
                    class="form-control"
                    id="montant"
                    name="montant"
                    min="1"
                    step="1"
                    placeholder="Ex : 5000"
                    value="<?= esc(old('montant')) ?>"
                    required
                >
            </div>
            <button type="submit" class="btn btn-success w-
100">Deposer</button>
        </form>
    </div>
</div>

</div>
<?= $this->endSection() ?>

```

app/Views/client/retrait.php

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<div class="row justify-content-center">
    <div class="col-md-6">

        <?php $error = session()->getFlashdata('error'); ?>
        <?php if ($error): ?>
            <div class="alert alert-danger"><?= esc($error) ?></div>
        <?php endif; ?>

        <div class="card shadow-sm">
            <div class="card-body p-4">
                <h1 class="h4 mb-3">Retrait</h1>

```

```

        <form method="post" action="<?= site_url('client/retrait') ?
    >">

        <?= csrf_field() ?>
        <div class="mb-3">
            <label for="montant" class="form-label">Montant (Ar)
        </label>

            <input
                type="number"
                class="form-control"
                id="montant"
                name="montant"
                min="1"
                step="1"
                placeholder="Ex : 5000"
                value="<?= esc(old('montant')) ?>"
                required
            >
        </div>
        <button type="submit" class="btn btn-warning w-
100">Retirer</button>
    </form>
</div>
</div>
<?= $this->endSection() ?>

```

app/Views/client/transfert.php (avec suggestion de numeros, voir ci-dessous)

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<div class="row justify-content-center">
    <div class="col-md-6">

        <?php $error = session()->getFlashdata('error'); ?>
        <?php if ($error): ?>
            <div class="alert alert-danger"><?= esc($error) ?></div>
        <?php endif; ?>

        <div class="card shadow-sm">
            <div class="card-body p-4">
                <h1 class="h4 mb-3">Transfert</h1>
                <form method="post" action="<?= site_url('client/transfert')
?>">

                    <?= csrf_field() ?>
                    <div class="mb-3">
                        <label for="destinataire" class="form-label">Numero
du destinataire</label>
                        <input
                            type="text"

```

```

        class="form-control"
        id="destinataire"
        name="destinataire"
        list="destinataires-suggestions"
        autocomplete="off"
        placeholder="Ex : 0331234567"
        value="<? = esc(old('destinataire')) ?>"
        required
    >
    <datalist id="destinataires-suggestions">
        <?php foreach ($suggestions as $numero) : ?>
            <option value="<? = esc($numero) ?>"></option>
        <?php endforeach; ?>
    </datalist>
</div>
<div class="mb-3">
    <label for="montant" class="form-label">Montant (Ar)

</label>

    <input
        type="number"
        class="form-control"
        id="montant"
        name="montant"
        min="1"
        step="1"
        placeholder="Ex : 5000"
        value="<? = esc(old('montant')) ?>"
        required
    >
</div>
<button type="submit" class="btn btn-primary w-
100">Transférer</button>
</form>
</div>
</div>
<? = $this->endSection() ?>

```

app/Views/client/historique.php

```

<? = $this->extend('layouts/main') ?>

<? = $this->section('content') ?>
<div class="row justify-content-center">
    <div class="col-md-10">
        <h1 class="h4 mb-3">Historique des operations</h1>

        <?php if (empty($transactions)): ?>
            <div class="alert alert-info">Aucune transaction.</div>
        <?php else: ?>

```



```

<div class="table-responsive">
  <table class="table table-striped table-hover">
    <thead class="table-dark">
      <tr>
        <th>Date</th>
        <th>Type</th>
        <th class="text-end">Montant</th>
        <th class="text-end">Frais</th>
        <th class="text-end">Solde avant</th>
        <th class="text-end">Solde apres</th>
      </tr>
    </thead>
    <tbody>
      <?php foreach ($transactions as $t): ?>
        <tr>
          <td><?= esc($t->created_at) ?></td>
          <td><?= esc($t->type_libelle) ?></td>
          <td class="text-end"><?= number_format($t-
>montant, 2, ',', ' ') ?></td>
          <td class="text-end"><?= number_format($t-
>frais, 2, ',', ' ') ?></td>
          <td class="text-end"><?= number_format($t-
>solde_avant, 2, ',', ' ') ?></td>
          <td class="text-end"><?= number_format($t-
>solde_apres, 2, ',', ' ') ?></td>
        </tr>
      <?php endforeach; ?>
    </tbody>
  </table>
</div>
<?php endif; ?>
</div>
</div>
<?= $this->endSection() ?>

```

Coordination (fichiers partagés déjà faits)

- **app/Config/Routes.php** — toutes les routes des deux lots sont déjà déclarées (groupes **opérateur/*** et **client/*** + **login/logout**). Chacun crée uniquement son controller avec les noms de méthode exacts listés ci-dessus ; en principe plus besoin de toucher ce fichier (sauf activation du filtre **clientAuth** par le Lot 2).
- **app/Views/layouts/main.php** — layout Bootstrap commun avec navbar (liens vers toutes les pages des deux lots) déjà créé. Chaque vue doit l'étendre, ne pas le modifier sans prévenir l'autre :

```

<?= $this->extend('layouts/main') ?>
<?= $this->section('content') ?>
    ... contenu de la page ...
<?= $this->endSection() ?>

```

- `app/Controllers/BaseController.php` — helpers `form` et `url` déjà actives pour tous les controllers (utilisables directement : `site_url()`, `form_open()`, etc.)
- `app/Config/App.php` — `indexPage` vidé pour des URLs propres (`/client` au lieu de `/index.php/client`)

TODO Version 2

Énoncé :

Coté opérateur

- Configuration des préfixes valable pour les autres opérateurs (ex: 032 et 031, ...)
- Configuration % en plus de commissions pour les transferts vers les autres opérateurs
- Sur la page "Situation gain via les différents frais", séparer opérateur et autres opérateurs
- Situation des montants à envoyer à chaque opérateur

Coté client

- Option inclure frais de retrait lors de l'envoi
- Envoi multiple vers plusieurs numéros (divisé le montant pour chaque numéro)

Précisions apportées après discussion (remplacent les recommandations par défaut précédentes) :

1. **Login en 2 étapes (nouveau, condition d'accès à tout le reste de la V2)** — au lieu d'un login unique cote client, l'accueil (/) devient un choix de role :
 - **Client** → flow inchangé (numero de telephone, restrictions cote client comme en V1)
 - **Opérateur** → acces complet a toutes les fonctionnalites/pages `operateur/*`

Actuellement (V1) les routes `operateur/*` n'ont **aucune protection** (juste caché du menu si pas connecte en tant que client, ce qui n'est pas une securite — n'importe qui peut appeler `/operateur/gains` directement). La V2 corrige ca avec un vrai filtre.

2. **Les préfixes "autres opérateurs" ne sont pas une table séparée** — on réutilise la table `prefixes` existante avec une colonne `status` (`principal` / `autre`). Le bouton "Ajouter" existant (V1) crée toujours un préfixe `principal` par défaut ; un nouveau choix (ex: case à cocher ou select "Autre opérateur") permet de créer un préfixe `autre`, avec alors un champ `pourcentage_commission` en plus.
3. **Un numéro externe n'a pas de compte chez nous** — on reconnait seulement son **préfixe** (`status = 'autre'`), pas le numéro individuel. On ne suit pas de solde par numéro externe, mais on accumule une **situation par opérateur externe** (regroupée par préfixe), nécessaire pour les transferts sortants.
4. **Commission ≠ frais de transfert** — les deux s'additionnent, mais ce ne sont pas la même chose :
 - `frais` (barème existant, table `frais`) = gain de **notre** opérateur, comme en V1
 - `commission` (% du préfixe externe) = part due a **l'autre** opérateur, ce n'est **pas** un gain pour nous — juste un montant qu'on doit leur reverser
5. **"Montants à envoyer à chaque opérateur" = montant brut (pour le destinataire) + commission (pour l'opérateur)**, les deux calculés séparément et à afficher côte à côte pour chaque opérateur

externe.

6. **Frais de retrait inclus (coché)** : le destinataire reçoit `montant_saisi + frais_de_retrait_estimé` (au lieu de juste `montant_saisi`). L'émetteur paie donc ce montant majoré, en plus du frais de transfert standard (et de la commission si transfert externe).
7. **Envoi multiple : frais calculé sur la part de chaque destinataire**, chaque envoi étant une transaction distincte avec son propre frais selon le barème.

Analyse & schéma DB (commun, à faire avant les 2 lots — comme pour la V1)

Toutes ces modifications se font sur les tables **existantes** (`prefixes`, `transactions`), pas de nouvelle table à créer — à appliquer dans `base.sql` :

```
-- Table prefixes : distinguer nos prefixes de ceux des autres operateurs
ALTER TABLE prefixes ADD COLUMN status VARCHAR(10) NOT NULL DEFAULT 'principal';
-- 'principal' ou 'autre'

ALTER TABLE prefixes ADD COLUMN pourcentage_commission DECIMAL(5,2) NOT NULL
DEFAULT 0;
-- utilise seulement si status = 'autre'

-- Table transactions : tracer les transferts externes et le detail des
majorations
ALTER TABLE transactions ADD COLUMN numero_externe VARCHAR(20);
-- rempli seulement si transfert vers un
prefixe 'autre'
-- (client_destination_id reste NULL dans
ce cas)
ALTER TABLE transactions ADD COLUMN commission DECIMAL(15,2) NOT NULL DEFAULT 0;
-- part due a l'autre operateur
(transferts externes uniquement)
ALTER TABLE transactions ADD COLUMN frais_retrait_inclus DECIMAL(15,2) NOT NULL
DEFAULT 0;
-- montant ajoute au transfert si l'option
"frais de
l'emetteur" a ete cochee par
```

Pas de migration séparée nécessaire pour l'instant : on éditera directement `base.sql` (le projet est encore en développement actif, pas encore en prod avec des vraies données à préserver).

Algorithme du transfert (V2) — remplace `TransactionModel::transfert()` de la V1

```
fonction transfert(clientId, numeroDestinataire, montantSaisi,
inclureFraisRetrait):

    destClient = ClientModel.findByNumero(numeroDestinataire)
```

```
si destClient existe:
    estExterne = false
sinon:
    prefixe = 3 premiers caracteres de numeroDestinataire
    prefixRow = PrefixModel.where(prefixe, status='autre').first()
    si prefixRow n'existe pas:
        erreur "numero ou prefixe non reconnu"
    estExterne = true
    pourcentageCommission = prefixRow.pourcentage_commission

fraisRetraitInclus = 0
si inclureFraisRetrait:
    fraisRetraitInclus = calculerFrais(type='retrait', montantSaisi)

montantCredite = montantSaisi + fraisRetraitInclus    // ce que le
destinataire recoit

fraisTransfert = calculerFrais(type='transfert', montantSaisi) // toujours,
notre gain
commission = estExterne ? (montantSaisi * pourcentageCommission / 100) : 0

totalDebit = montantCredite + fraisTransfert + commission

si emetteur.solde < totalDebit:
    erreur "solde insuffisant"

debiter emetteur de totalDebit
si !estExterne:
    crediter destClient de montantCredite
// si externe : personne chez nous n'est credite, on doit juste ce montant a
l'autre operateur

insérer transactions (
    type_operation_id = transfert,
    client_id = emetteurId,
    client_destination_id = estExterne ? null : destClient.id,
    numero_externe = estExterne ? numeroDestinataire : null,
    montant = montantCredite,
    frais = fraisTransfert,
    commission = commission,
    frais_retrait_inclus = fraisRetraitInclus,
    solde_avant = emetteur.solde,
    solde_apres = emetteur.solde - totalDebit
)
```

Lot 1 — Côté Opérateur (V2) (ETU003968)

Fichiers à créer / modifier :

app/Filters/OperateurAuthFilter.php	(nouveau, protege operateur/*)
app/Controllers/OperateurController.php	(login()/attempt() +
prefixes())/storePrefixe()	
	a adapter pour le champ
status+commission +	
	gains() a modifier + nouvel ecran)
app/Views/operateur/login.php	(nouveau)
app/Views/operateur/prefixes.php	(a modifier : champ status +
commission)	
app/Views/operateur/gains.php	(a modifier : 2 sections)
app/Views/operateur/situation_operateurs.php	(nouveau)
app/Config/Routes.php	(routes a ajouter/protéger)
app/Config/Filters.php	(alias operateurAuth)
app/Views/layouts/main.php	(menu Operateur conditionne par
is_operateur)	

Checklist et code :

- ☐ Migration `base.sql` : colonnes `status/pourcentage_commission` sur `prefixes`, colonnes `numero_externe/commission/frais_retrait_inclus` sur `transactions` (voir schema ci-dessus)
- ☐ `OperateurController::login()` (GET) / `attempt()` (POST) — verifie un mot de passe unique (stocke dans `.env`, ex `operateur.password`, comparaison simple), stocke `session()` - `>set('is_operateur', true)`, redirige vers `operateur/prefixes`. (Pas de table d'utilisateurs `operateur` — un seul role "operateur", pas de comptes nominatifs, coherent avec l'esprit "pas d'inscription" du projet. A confirmer/ajuster si vous voulez plusieurs comptes operateur nommes.)
- ☐ `OperateurAuthFilter` — verifie `session()->get('is_operateur')`, sinon redirige vers `operateur/login`
- ☐ Dans `Routes.php` : sortir `operateur/login` du groupe `protege`, wrapper le reste du groupe `operateur` avec `['filter' => 'operateurAuth']` (meme principe que `clientAuth` pour le Lot 2 en V1)
- ☐ Dans `Filters.php` : ajouter l'alias `'operateurAuth' => \App\Filters\OperateurAuthFilter::class`
- ☐ Modifier `OperateurController::prefixes()/storePrefixe()` : le formulaire d'ajout propose un choix "Principal" (par default, comportement V1 inchange) ou "Autre operateur" (avec champ `%commission` affiche seulement si "Autre" est choisi) ; la liste affiche le `status` de chaque prefixe
- ☐ Modifier `OperateurController::gains()` — 2 sections distinctes : 1. **Gains de l'operateur** (inchange par rapport a la V1) : `SUM(frais)` par type d'operation (depot/retrait/transfert, interne ET externe confondus car le frais standard nous revient toujours) 2. **Montants dus aux autres operateurs** (nouveau) : par prefixe externe (`transactions.numero_externe` jointe sur son prefixe), `SUM(montant)` (brut, a transmettre) + `SUM(commission)` (leur part) + total (brut+commission a reverser)
- ☐ Nouvel ecran `OperateurController::situationOperateurs()` — meme requete que la section 2 de `gains()` ci-dessus (peut reutiliser la meme methode/vue, ou etre une page dediee si vous preferez separer visuellement "nos gains" de "ce qu'on doit aux autres" — a trancher selon preference d'UX, le calcul est identique)
- ☐ Route `operateur/situation-operateurs` (GET) si ecran separe
- ☐ `layouts/main.php` : menu "Operateur" visible si `session()->get('is_operateur')` (au lieu de `client_id` actuellement — c'est un bug herite de la V1 a corriger), bouton Login/Logout qui gere les 2

roles

Lot 2 — Côté Client (V2) (ETU004311) Termine

Fichiers créés / modifiés :

```

app/Views/home.php                (choix Client / Operateur)
app/Models/TransactionModel.php    (transfert() reecrit, transfertMultiple()
ajoute)
app/Controllers/ClientController.php (storeTransfert() +
transfertMultiple()/storeTransfertMultiple())
app/Views/client/transfert.php      (checkbox frais de retrait inclus + lien
envoi multiple)
app/Views/client/transfert_multiple.php (nouveau)
app/Config/Routes.php               (client/transfert-multiple)

```

Checklist et code :

- ☒ `app/Views/home.php` — choix "Je suis client" (→ `login`) / "Je suis l'opérateur" (→ `operateur/login`)

```

<?= $this->extend('layouts/main') ?>

<?= $this->section('content') ?>
<div class="d-flex flex-column justify-content-center align-items-center"
style="min-height: 60vh;">
    <div class="card shadow-sm" style="max-width: 480px; width: 100%;">
        <div class="card-body p-4 text-center">
            <h1 class="h4 mb-3">Mobile Money</h1>
            <p class="text-muted mb-4">Simulation d'opérateur Mobile Money.
Qui êtes-vous ?</p>
            <div class="d-flex flex-column gap-2">
                <a href="<?= site_url('login') ?>" class="btn btn-primary">Je
suis client</a>
                <a href="<?= site_url('operateur/login') ?>" class="btn btn-
dark">Je suis l'opérateur</a>
            </div>
        </div>
    </div>
</div>
<?= $this->endSection() ?>

```

- ☒ `TransactionModel::transfert()` réécrit selon l'algorithme détaillé plus haut : gère un client interne existant OU un numéro dont le préfixe est reconnu `autre` (`PrefixModel::findAutrePrefixe()`), calcule le frais standard + la commission externe + la majoration "frais de retrait inclus", débite l'émetteur du total, ne crédite personne chez nous si externe (pas de compte à créditer)

- ☒ `TransactionModel::transfertMultiple()` — divise le montant total à parts égales (dernier destinataire = reliquat d'arrondi), appelle `transfert()` en boucle, englobé dans une seule transaction SQL (les transactions CI4 se composent par profondeur — `transStart()/transComplete()` imbriqués fonctionnent nativement), rollback explicite si un envoi échoue en cours de route

`app/Models/TransactionModel.php` (méthodes modifiées/ajoutées)

```
public function transfert(int $clientId, string $numeroDestinataire, float
$montantSaisi, bool $inclureFraisRetrait = false): array
{
    $clientModel = model(ClientModel::class);
    $prefixeModel = model(PrefixeModel::class);

    $destinataire = $clientModel->findByNumero($numeroDestinataire);
    $estExterne = $destinataire === null;
    $pourcentageCommission = 0.0;

    if ($estExterne) {
        $prefixeExterne = $prefixeModel-
>findAutrePrefixe($numeroDestinataire);

        if ($prefixeExterne === null) {
            throw new \RuntimeException('Numero ou prefixe non reconnu.');
        }

        $pourcentageCommission = (float)
$prefixeExterne['pourcentage_commission'];
    }

    $this->db->transStart();

    $emetteur = $clientModel->find($clientId);
    $typeId = $this->getCodeId('transfert');

    $fraisRetraitInclus = $inclureFraisRetrait
        ? $this->calculerFrais($this->getCodeId('retrait'), $montantSaisi)
        : 0.0;

    $montantCredite = $montantSaisi + $fraisRetraitInclus;
    $fraisTransfert = $this->calculerFrais($typeId, $montantSaisi);
    $commission = $estExterne ? round($montantSaisi *
$pourcentageCommission / 100, 2) : 0.0;

    $totalDebit = $montantCredite + $fraisTransfert + $commission;

    if ($emetteur->solde < $totalDebit) {
        $this->db->transRollback();
        throw new \RuntimeException('Solde insuffisant pour le transfert.');
    }

    $soldeEmetteurApres = $emetteur->solde - $totalDebit;
    $clientModel->update($clientId, ['solde' => $soldeEmetteurApres]);
}
```

```

        if (! $estExterne) {
            $clientModel->update($destinataire->id, ['solde' => $destinataire->solde + $montantCredite]);
        }

        $this->insert([
            'type_operation_id' => $typeId,
            'client_id' => $clientId,
            'client_destination_id' => $estExterne ? null : $destinataire->id,
            'numero_externe' => $estExterne ? $numeroDestinataire : null,
            'montant' => $montantCredite,
            'frais' => $fraisTransfert,
            'commission' => $commission,
            'frais_retrait_inclus' => $fraisRetraitInclus,
            'solde_avant' => $emetteur->solde,
            'solde_apres' => $soldeEmetteurApres,
        ]);

        $this->db->transComplete();

        return [
            'frais' => $fraisTransfert,
            'commission' => $commission,
            'solde' => $soldeEmetteurApres,
            'est_externe' => $estExterne,
        ];
    }

    public function transfertMultiple(int $clientId, array $numeros, float $montantTotal, bool $inclureFraisRetrait = false): array
    {
        $nombreDestinataires = count($numeros);

        if ($nombreDestinataires === 0) {
            throw new \RuntimeException('Aucun destinataire.');
        }

        $part = floor(($montantTotal / $nombreDestinataires) * 100) / 100;
        $resultats = [];

        $this->db->transStart();

        try {
            foreach ($numeros as $index => $numero) {
                $montantPart = ($index === $nombreDestinataires - 1)
                    ? round($montantTotal - ($part * ($nombreDestinataires - 1)),
2)
                    : $part;

                $resultats[] = $this->transfert($clientId, $numero, $montantPart, $inclureFraisRetrait);
            }
        } catch (\RuntimeException $e) {

```



```

        $this->db->transRollback();
        throw $e;
    }

    $this->db->transComplete();

    return $resultats;
}

```

- ☒ `ClientController::storeTransfert()` — lit `inclure_frais_retrait`, passe le numéro brut à `TransactionModel::transfert()` (plus besoin que le destinataire existe déjà comme client), message de confirmation incluant la commission si transfert externe
- ☒ `ClientController::transfertMultiple()` (GET) / `storeTransfertMultiple()` (POST) — valide au moins 2 destinataires, rejette l'auto-transfert, delegue a `TransactionModel::transfertMultiple()`

`app/Controllers/ClientController.php` (méthodes modifiées/ajoutées)

```

public function storeTransfert()
{
    $destinataire      = trim($this->request->getPost('destinataire') ??
    '');
    $montant           = (float) $this->request->getPost('montant');
    $inclureFraisRetrait = (bool) $this->request->
    >getPost('inclure_frais_retrait');

    if ($destinataire === '') {
        return redirect()->back()->with('error', 'Veuillez saisir le numero
    du destinataire.');
```

```

    }

    if ($montant <= 0) {
        return redirect()->back()->with('error', 'Le montant doit etre
    superieur a 0.');
```

```

    }

    $clientModel = new ClientModel();
    $moi         = $clientModel->find($this->clientId());

    if ($destinataire === $moi->numero) {
        return redirect()->back()->with('error', 'Vous ne pouvez pas vous
    transférer a vous-meme.');
```

```

    }

    $transactionModel = new TransactionModel();

    try {
        $resultat = $transactionModel->transfert($this->clientId(),
    $destinataire, $montant, $inclureFraisRetrait);
    } catch (\RuntimeException $e) {

```

```
        return redirect()->back()->with('error', $e->getMessage());
    }

    $message = 'Transfert de ' . number_format($montant, 0, ',', ' ') . '
effectue (frais : ' . number_format($resultat['frais'], 0, ',', ' ') . ')';

    if ($resultat['commission'] > 0) {
        $message .= ', commission autre operateur : ' .
number_format($resultat['commission'], 0, ',', ' ');
    }

    return redirect()->to(site_url('client'))->with('success', $message .
'.');
}

public function transfertMultiple()
{
    return view('client/transfert_multiple');
}

public function storeTransfertMultiple()
{
    $numeros          = array_values(array_filter(array_map('trim', $this-
>request->getPost('numeros') ?? [])));
    $montant          = (float) $this->request->getPost('montant');
    $inclureFraisRetrait = (bool) $this->request-
>getPost('inclure_frais_retrait');

    if (count($numeros) < 2) {
        return redirect()->back()->with('error', 'Veuillez saisir au moins 2
destinataires.');
```

```
    }

    if ($montant <= 0) {
        return redirect()->back()->with('error', 'Le montant total doit etre
superieur a 0.');
```

```
    }

    $clientModel = new ClientModel();
    $moi          = $clientModel->find($this->clientId());

    if (in_array($moi->numero, $numeros, true)) {
        return redirect()->back()->with('error', 'Vous ne pouvez pas vous
transférer a vous-meme.');
```

```
    }

    $transactionModel = new TransactionModel();

    try {
        $transactionModel->transfertMultiple($this->clientId(), $numeros,
$montant, $inclureFraisRetrait);
    } catch (\RuntimeException $e) {
        return redirect()->back()->with('error', $e->getMessage());
    }
}
```

```

        return redirect()->to(site_url('client'))->with(
            'success',
            'Envoi multiple de ' . number_format($montant, 0, ',', ' ') . '
reparti entre ' . count($numeros) . ' destinataires effectue.'
        );
    }

```

- ☒ `client/transfert.php` — checkbox "Inclure les frais de retrait" + lien vers l'envoi multiple ; `client/transfert_multiple.php` (nouveau) — liste dynamique de destinataires (ajout/retrait en JS vanilla) + montant total + meme checkbox
- ☒ Routes `client/transfert-multiple` (GET/POST) ajoutées dans le groupe `client` (protege par `clientAuth`, comme le reste)

Testé en HTTP (bout en bout), verification par calcul manuel :

- Depot 200 000 → transfert interne 5 000 (frais 50) → transfert externe 5 000 vers prefixe `autre` 032 (frais 50 + commission 100) → transfert vers prefixe non reconnu (rejete, aucune transaction creee) → transfert interne 5 000 avec frais de retrait inclus (destinataire credite de 5 050) → envoi multiple 30 000 vers 2 destinataires (15 000 chacun, un interne + un externe)
- **Conservation de l'argent verifiee** : 200 000 deposes = 179 050 restant sur les comptes clients + 20 000 partis en externe (montant brut) + 550 de frais (notre gain)
 - 400 de commission due aux autres operateurs = 200 000 exactement

Definition of Done — V2

- ☒ Login en 2 étapes fonctionnel, `operateur/*` réellement protégé (testé en accédant directement à une URL operateur sans être connecté)
- ☒ Schéma DB V2 appliqué (`prefixes.status/pourcentage_commission`, `transactions.numero_externe/commission/frais_retrait_inclus`)
- ☒ Ajout d'un préfixe "Autre opérateur" avec commission fonctionnel
- ☒ Transfert vers un numéro externe fonctionnel (montant brut au destinataire virtuel + frais standard chez nous + commission due à l'autre opérateur)
- ☒ Page gains séparée : nos gains (frais) / montants dus aux autres opérateurs (brut + commission)
- ☒ Option "frais de retrait inclus" fonctionnelle (vérifiée avec calcul manuel)
- ☒ Envoi multiple fonctionnel (division + arrondi correct, frais par destinataire)
- ☐ Tag Git `v2` posé sur `main`